

CODESYS Without HMI Only

Minimal sub-package — remote setpoint cabinet control, CODESYS + Modbus only

Author: OJ (Omkar Joshi) — Oliver Mechatronics **Target:** Watlow F4S (SN 038983), Modbus RTU / RS-232, `/dev/ttyWatlowF4S` → COM2 **Parent package:** `docs/DEPLOYMENT_AND_TEST.md` (this is a strict subset of it) **Scope:** Prove the Modbus layer inside CODESYS with **zero HMI**. No WebVisu, no GVL_HMI, no FB, no logo, no dialogs. Everything is operated from the CODESYS **watch window**.

Why this package exists: the full package couples the Modbus layer to a WebVisu HMI. If Modbus is broken, HMI debugging masks it. This package removes every non-essential layer so a failure can only be Modbus, wiring, or F4S state — nothing else.

1. Definition of Done (3 items only)

#	Requirement	Proven by
1	Read current cabinet temperature in CODESYS	<code>rChamberTemp</code> tracks F4S front panel ± 0.1 °C
2	Read current setpoint in CODESYS	<code>rConfirmedSetpoint</code> matches F4S SP1
3	Write a new setpoint from CODESYS, confirmed	Write → F4S SP1 changes → read-back matches

Explicitly **out of scope here:** WebVisu, GVL_HMI, FB_CabinetSetpointControl, logo/branding, operator SOP, EL1409 over-temp DI, fault banners. Those return in the full package.

2. Object inventory — exactly what must exist

Delete/ignore anything not on this list while commissioning.

Object	Type	Purpose
E_SetpointState	DUT (enum)	State machine states
E_FaultCode	DUT (enum)	Fault reasons
PLC_PRG	POU (Program)	All logic. Assigned to MainTask
RAW_Actual_Temp	WORD (I/O-mapped)	Reg 100 raw, ×10
RAW_Actual_Setpoint	WORD (I/O-mapped)	Reg 300 raw read-back, ×10
RAW_Target_Setpoint	WORD (I/O-mapped)	Reg 300 raw write, ×10
xWriteSetpointTrigger	BOOL (I/O-mapped)	FC06 one-shot trigger

GVL_Modbus is not required in this package. If it exists, leave it — but nothing here references it. Its old **xwriteTrigger** was orphaned (mapped to nothing), which is why writes silently did nothing.

3. Linux layer (already proven — verify only)

```
bash
ls -la /dev/ttyWatlowF4S          # -> ttyUSB0 (stable udev symlink)
cat /etc/CODESYSControl_User.cfg  # -> Linux.Devicefile.2=/dev/ttyWatlowF4S, po
sudo systemctl restart codesyscontrol
```

Baseline sanity (must pass before touching CODESYS):

```
bash
mbpoll -m rtu -a 1 -b 19200 -P none -t 4 -r 100 -c 1 -1 -0 /dev/ttyWatlowF4S
mbpoll -m rtu -a 1 -b 19200 -P none -t 4 -r 300 -c 1 -1 -0 /dev/ttyWatlowF4S
```

Port contention: never leave **mbpoll** running while CODESYS is online. One process owns the serial device at a time. Bench-test, exit, *then* go online.

4. Device tree — exact settings

```
Device (CODESYS Control for Linux ARM64 SL)
└─ Modbus_COM                                COM2 · 19200 · None · 8 · 1
    └─ Modbus_Master_COM_Port                RTU · timeout 1000 ms · frame gap 50 ms
        |                                     Bus cycle task = MainTask (explicit, NOT
"parent")
    └─ Watlow_F4S (slave)                    address 1 · timeout 1000 ms
```

EtherCAT master: disabled for this package (its bus cycle task is pinned to EtherCAT_Task so it cannot contend with MainTask).

Channels — all three are HOLDING registers

#	Name	Access type	FC	Offset	Len	Trigger
0	ReadChamberTemp	Read Holding Registers	03	100	1	Cyclic 1000 ms
1	ReadSetpoint	Read Holding Registers	03	300	1	Cyclic 1000 ms
2	WriteSetpoint	Write Single Register	06	300	1	Application (rising edge)

CH1 correction — this is a real bug fix. If ReadSetpoint is set to *Read **Input** Registers*, it issues **FC04**, which addresses a completely different memory space on the F4S. There is nothing there. It will never return data no matter how correct everything else is. Every proven `mbpoll` read used `-t 4` = **holding** register table. Set it to **Read Holding Registers**.

CH2 must stay Application-triggered, never cyclic. Register 300 is EEPROM-backed on the F4S (~100k write cycles). A cyclic FC06 would rewrite it every second and destroy the EEPROM within days. One write per operator request — that is the entire point of the edge trigger.

I/O Mapping

Channel	Address	Variable	Type
ReadChamberTemp[0]	<code>%IW82</code>	<code>RAW_Actual_Temp</code>	WORD
ReadSetpoint[0]	<code>%IW83</code>	<code>RAW_Actual_Setpoint</code>	WORD

Channel	Address	Variable	Type
Write_Setpoint[0]	%QW1	RAW_Target_Setpoint	WORD
Write_Setpoint trigger	(bool row)	xWriteSetpointTrigger	BOOL

Always update variables = Enabled 1.

The trigger row is the one everyone misses. Expand the Write_Setpoint node with the small arrow. Beneath the %QW1 data row there is a **second, boolean row** — the trigger. If it is unmapped, the data value stages correctly but **nothing ever tells the channel to send it**. Writes appear to succeed and do nothing. Map it to xWriteSetpointTrigger.

5. PLC_PRG — complete, final

```
st
```

```

PROGRAM PLC_PRG
VAR
    rChamberTemp      : REAL;
    rReqSetpoint       : REAL := 26.5;    (* set this in the watch window *)
    rConfirmedSetpoint : REAL;

    rMinSetpoint       : REAL := 0.0;
    rMaxSetpoint       : REAL := 200.0;

    eSetpointState     : E_SetpointState := E_SetpointState.IDLE;
    eFaultCode         : E_FaultCode := E_FaultCode.NO_FAULT;

    xStartWrite        : BOOL := FALSE;   (* toggle this in the watch window *)
    xPrevStartWrite    : BOOL := FALSE;
    xWriteTriggered    : BOOL := FALSE;

    dwWriteTimeout     : DWORD := 0;
    dwMaxTimeout       : DWORD := 300;    (* ~3 s at 10 ms MainTask *)
END_VAR

(* --- 1. Scale raw -> engineering units (F4S carries one implied decimal) ---
rChamberTemp      := RAW_Actual_Temp / 10.0;
rConfirmedSetpoint := RAW_Actual_Setpoint / 10.0;

(* --- 2. Rising edge on the manual test trigger --- *)
xWriteTriggered := (xStartWrite AND NOT xPrevStartWrite);
xPrevStartWrite := xStartWrite;

(* --- 3. State machine --- *)
CASE eSetpointState OF

    E_SetpointState.IDLE:
        xWriteSetpointTrigger := FALSE;
        dwWriteTimeout := 0;
        IF xWriteTriggered THEN
            eSetpointState := E_SetpointState.READY;
        END_IF

    E_SetpointState.READY:
        IF rReqSetpoint < rMinSetpoint THEN
            eFaultCode := E_FaultCode.RANGE_LOW;
            eSetpointState := E_SetpointState.FAULTED;
        ELSIF rReqSetpoint > rMaxSetpoint THEN

```

```

        eFaultCode := E_FaultCode.RANGE_HIGH;
        eSetpointState := E_SetpointState.FAULTED;
    ELSE
        RAW_Target_Setpoint := DWORD_TO_WORD(REAL_TO_DWORD(rReqSetpoint * 1
        eSetpointState := E_SetpointState.WRITING;
    END_IF

E_SetpointState.WRITING:
    xWriteSetpointTrigger := TRUE;    (* one-scan pulse -> FC06 fires *)
    dwWriteTimeout := 0;
    eSetpointState := E_SetpointState.CONFIRM;

E_SetpointState.CONFIRM:
    xWriteSetpointTrigger := FALSE;    (* drop immediately to re-arm the edge
    dwWriteTimeout := dwWriteTimeout + 1;

    IF ABS(rConfirmedSetpoint - rReqSetpoint) < 0.1 THEN
        eFaultCode := E_FaultCode.NO_FAULT;
        eSetpointState := E_SetpointState.IDLE;
    ELSIF dwWriteTimeout > dwMaxTimeout THEN
        eFaultCode := E_FaultCode.NOT_ACCEPTED;
        eSetpointState := E_SetpointState.FAULTED;
    END_IF

E_SetpointState.FAULTED:
    xWriteSetpointTrigger := FALSE;
    IF xWriteTriggered THEN
        eFaultCode := E_FaultCode.NO_FAULT;
        eSetpointState := E_SetpointState.READY;
    END_IF

END_CASE

```

Design notes:

- `xStartWrite` (manual, unmapped) is deliberately separate from `xWriteSetpointTrigger` (the real hardware bit). The *state machine* decides when hardware fires — not the operator. This is what guarantees exactly one FC06 per request.
- Timeout raises `NOT_ACCEPTED`, not `COMMS_TIMEOUT`. Read-back mismatch after a successful send means the F4S *heard* it and *refused* it — almost always the menu-state issue (§7). True comms loss shows up as reads freezing, which is a different symptom.

6. Commissioning sequence

1. Correct CH1 to **Read Holding Registers**.
 2. Confirm Modbus_COM baud = **19200** (not 9600).
 3. Map the **trigger boolean row** to `xWriteSetpointTrigger`.
 4. Confirm Master bus cycle task = **MainTask** (explicit).
 5. `Build → Rebuild All` → expect **0 errors**.
 6. Download → **F5** (Start) → status bar = RUNNING.
 7. Device tree: Modbus_COM, Master, Watlow_F4S all **green**.
 8. Watch window — add: `rChamberTemp`, `rConfirmedSetpoint`, `rReqSetpoint`, `xStartWrite`, `eSetpointState`, `eFaultCode`, `RAW_Actual_Temp`, `RAW_Actual_Setpoint`.
-

7. ⚠ F4S MENU STATE — read before any write test

The F4S silently rejects FC06 writes while its setpoint-edit screen is open. The write is acknowledged at the comms layer (`Written 1 references`) but never applied. Read-back returns the *old* value. There is no error — this is F4S firmware behaviour, confirmed on this unit.

Before every write test: press EXIT/ESCAPE on the F4S until the main run page is showing.

This single behaviour accounts for the majority of "the write says it worked but nothing happened" reports on this project.

8. Test plan (T1-T6)

Mark PASS only after **two consecutive** clean runs.

#	Test	Method	Pass criteria
T1	Temp read	Compare <code>rChamberTemp</code> to F4S panel	Match ± 0.1 °C, updates ~1 s
T2	Setpoint read	Compare <code>rConfirmedSetpoint</code> to F4S SP1	Exact match
T3	Setpoint write	F4S on main page . Set <code>rReqSetpoint := 26.5</code> , toggle <code>xStartWrite</code> TRUE	F4S SP1 $\rightarrow$ 26.5 °C in 1–2 s
T4	Write confirm	Watch after T3	State returns to <code>IDLE</code> , <code>eFaultCode = NO_FAULT</code> , <code>rConfirmedSetpoint = 26.5</code>
T5	Range reject	Set <code>rReqSetpoint := 250.0</code> , toggle <code>xStartWrite</code>	No write issued. <code>eFaultCode = RANGE_HIGH</code> , state <code>FAULTED</code>
T6	Menu-state reject	F4S inside setpoint-edit screen. Repeat T3	State <code>FAULTED</code> , <code>eFaultCode = NOT_ACCEPTED</code> — proves confirm logic actually catches a silent rejection

T6 is the most valuable test in this package. It is the only one that proves the read-back confirmation is real and not just optimistically reporting success.

9. Troubleshooting

Symptom	Cause	Fix
Master/slave red, no tooltip	Bus cycle task not explicitly MainTask	Set it explicitly; rebuild; download
<code>%IW</code> shown struck-through	Type mismatch — array vs scalar	Expand node, map element <code>[0]</code> , not the array
Reads work, <code>rConfirmedSetpoint</code> always 0	CH1 using FC04 (Input Registers)	Change to Read Holding Registers

Symptom	Cause	Fix
Write does nothing, no fault	Trigger boolean row unmapped	Map it to <code>xWriteSetpointTrigger</code>
Write does nothing, <code>NOT_ACCEPTED</code>	F4S in setpoint-edit menu	Exit to main run page (§7)
CODESYS can't open port	<code>mbpoll</code> still holding it	<code>sudo lsof /dev/ttyWatlowF4S</code> → kill → restart runtime
Everything green, values frozen	Baud 9600 vs F4S 19200	Set Modbus_COM baud to 19200

10. Exit criteria → return to full package

All of T1-T6 pass twice consecutively. At that point the Modbus layer is **proven inside CODESYS**, and `docs/DEPLOYMENT_AND_TEST.md` can be resumed from §4 (WebVisu) with confidence that any new failure is HMI-layer, not Modbus-layer.

Carry forward to the full package:

- `GVL_HMI` + `FB_CabinetSetpointControl` wrap this same logic; `xStartWrite` is replaced by `GVL_HMI.xSetBtn` from the WebVisu dialog. Nothing else changes.
- Open item for TL:** operating range. This package uses **0-200 °C** (matches everything bench-tested). The FB spec in the parent package says **30-130 °C**. These differ and one is wrong — confirm before production.

Discipline: Rebuild → Retest → Requalify → Repeat. No step skipped.